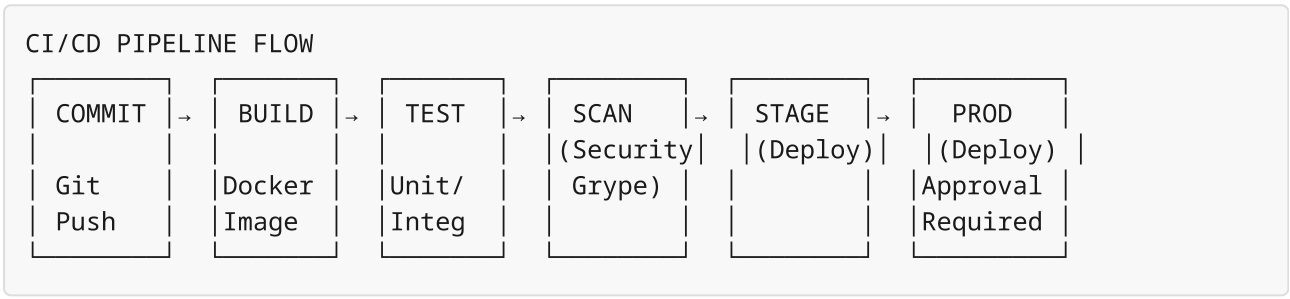
Part 2 — CI/CD & Azure DevOps

Chapter 6: CI/CD Fundamentals

Probability: 🔴 95%

Core Concepts



Top Interview Questions

Q1: "What is the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment?"

Term	Definition	Key Difference
CI	Automatically build + test on every commit	Merge to main frequently
Continuous Delivery	CI + automatically deploy to staging. Prod requires MANUAL approval	Human gate before prod
Continuous Deployment	CI + automatically deploy to ALL environments including prod	No human gate — fully automated

> ***Your answer:*** *"We practice Continuous Delivery. Every commit triggers build, test, and security scan. Staging deploys are automatic, but production requires manual approval through our change management process in Freshservice."*

Q2: "How do you handle rollbacks in your pipeline?"

> *"We use Kubernetes rolling updates with `maxUnavailable: 0` so old pods stay running until new ones pass readiness probes. For rollback, we use `kubectl rollout undo deployment/<name>`. Our Docker images are tagged with Git SHA, so we can always redeploy a known-good version. For database changes, we ensure backward compatibility — schema migrations are always additive first."*

Q3: "What is pipeline-as-code and why does it matter?"

> *"Pipeline-as-code means defining your CI/CD pipeline in a version-controlled file (Jenkinsfile, .gitlab-ci.yml, azure-pipelines.yml) rather than configuring through a UI. Benefits: versioned, reviewable, reproducible, auditable, and it enables GitOps practices."*

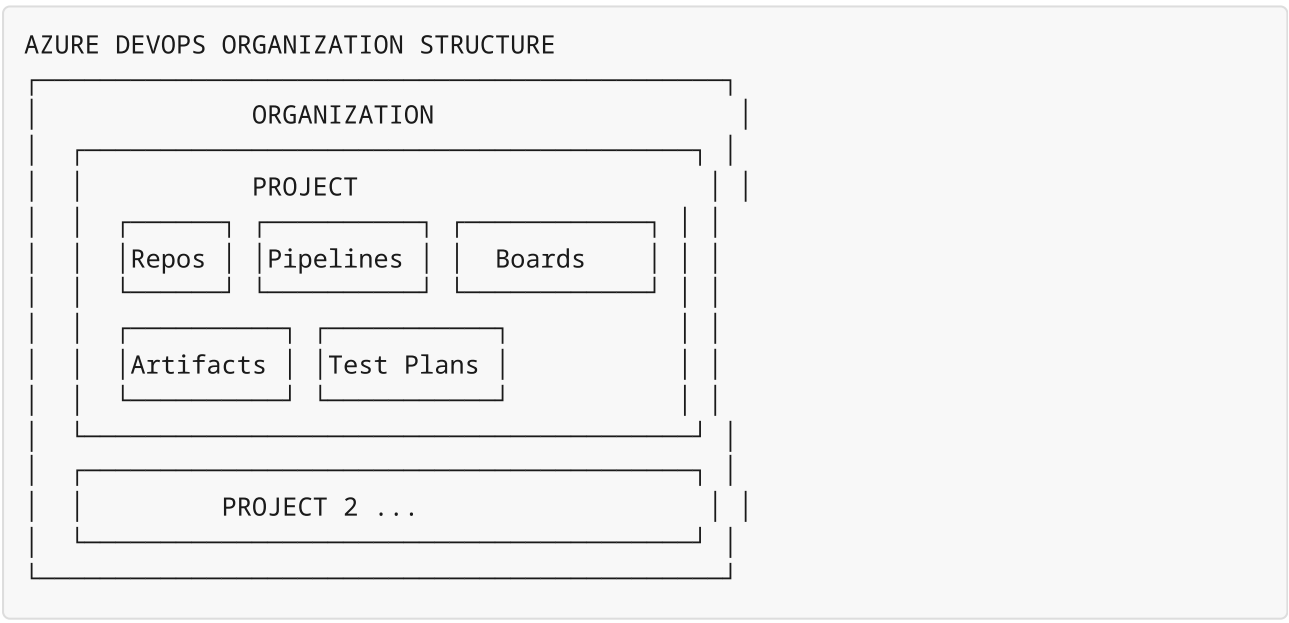
Chapter 7: Azure DevOps Overview

Probability: 🟠 85%

Mapping Azure DevOps to Your Experience

Azure DevOps Service	Your Equivalent	Key Differences
Azure Repos	Self-hosted GitLab	Git-based, similar. Azure Repos supports TFVC too
Azure Pipelines	Jenkins + GitLab CI	YAML-based, hosted agents, deeper Azure integration
Azure Boards	Jira	Work items, sprints, Kanban boards
Azure Artifacts	Nexus / JFrog Artifactory	npm, Maven, NuGet, pip, universal packages
Azure Test Plans	Manual QA tools	Exploratory + automated test management

Architecture



Key Interview Questions

Q: "What are the main components of Azure DevOps?"

> *"Azure DevOps is a suite of 5 services: Azure Repos for source control, Azure Pipelines for CI/CD, Azure Boards for project tracking, Azure Artifacts for package management, and Azure Test Plans for testing.*

> *These can be used independently or together. Think of it as GitLab's all-in-one approach — GitLab combines SCM, CI/CD, issues, and registry similarly."*

Q: "Azure DevOps vs GitHub Actions — when would you choose each?"

> *"Azure DevOps is better for enterprise environments needing boards, test plans, TFVC support, and complex multi-stage release pipelines with approvals. GitHub Actions is leaner, better for open-source and simpler workflows. For Infosys consulting projects, Azure DevOps is typically preferred because of its enterprise governance features."*

Q: "What's the difference between Classic and YAML pipelines?"

Feature	Classic (UI)	YAML
Definition	GUI-based, click-to-configure	Code-based, version-controlled
Version Control	Not tracked in Git	Lives in repo, full Git history
Reusability	Limited (task groups)	Templates, extends, parameters
Best For	Quick prototyping, legacy	Production, enterprise, audit
Recommendation	Avoid for new projects	**Always use YAML**

Chapter 8: Azure Repos

Probability: 🟡 55%

Mapping to GitLab

Feature	Azure Repos	GitLab	Notes
Git hosting	✅	✅	Functionally identical
Branch policies	✅ (UI-based)	✅ (Protected branches)	Azure: require reviewers, linked work items, successful build
Pull Requests	✅	✅ (Merge Requests)	Same concept, different name
Code search	✅	✅	
TFVC support	✅	❌	Legacy. Never recommend TFVC
Fork workflow	✅	✅	

Key Interview Questions

Q: "How do you enforce code quality in Azure Repos?"

> *"Through Branch Policies on the main branch: 1) Require minimum number of reviewers (usually 2), 2) Require linked work item for traceability, 3) Require successful build validation (pipeline must pass before merge), 4) Require comment resolution, 5) Optionally require specific reviewers for certain paths (CODEOWNERS equivalent)."*

Q: "What branching strategy do you recommend?"

> *"For most teams, I recommend trunk-based development with short-lived feature branches. Feature branches should live no more than 1-2 days. This reduces merge conflicts and enables CI. For larger teams or regulated environments, GitFlow with develop/release branches may be appropriate, but it adds complexity."*

Chapter 9: Azure Pipelines & YAML

Probability: 🟠 85%

Pipeline Structure

```yaml
# azure-pipelines.yml — Complete Example
trigger:                          # WHEN to run
  branches:
    include: [main, develop]
  paths:
    exclude: ['docs/*']

pool:                              # WHERE to run
  vmImage: 'ubuntu-latest'        # Microsoft-hosted agent
  # OR
  # name: 'SelfHostedPool'        # Self-hosted agent

variables:                         # Configuration
  dockerRegistry: 'myacr.azurecr.io'
  imageName: 'myapp'
  tag: '$(Build.BuildId)'

stages:                            # Pipeline stages
- stage: Build
  displayName: 'Build & Test'
  jobs:
  - job: BuildJob
    steps:
    - task: Docker@2               # Build Docker image
      inputs:
        command: 'build'
        repository: '$(imageName)'
        dockerfile: 'Dockerfile'
```

```
        tags: '$(tag)'

    - script: |                    # Run tests
        docker run --rm $(imageName):$(tag) npm test
      displayName: 'Run Unit Tests'

- stage: Deploy_Staging
  displayName: 'Deploy to Staging'
  dependsOn: Build
  condition: succeeded()
  jobs:
  - deployment: DeployStaging      # deployment job (not regular job)
    environment: 'staging'         # links to Azure DevOps Environment
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@0
            inputs:
              action: 'deploy'
              manifests: 'k8s/staging/*.yml'

- stage: Deploy_Production
  displayName: 'Deploy to Production'
  dependsOn: Deploy_Staging
  condition: succeeded()
  jobs:
  - deployment: DeployProd
    environment: 'production'      # Has manual approval gate
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@0
            inputs:
              action: 'deploy'
              manifests: 'k8s/prod/*.yml'
```

Jenkins vs Azure Pipelines — Side by Side

```
JENKINS (Jenkinsfile)                AZURE PIPELINES (YAML)
─────────────────────                ──────────────────────
pipeline {                           trigger:
  agent any                            branches: [main]
                                     pool:
                                       vmImage: 'ubuntu-latest'
  stages {                           stages:
    stage('Build') {                 - stage: Build
      steps {                          jobs:
        sh 'docker build .'           - job: BuildJob
      }                                  steps:
    }                                    - script: docker build .
    stage('Deploy') {                - stage: Deploy
      steps {                          jobs:
        sh 'kubectl apply ...'         - deployment: DeployJob
```

```
      }                                  environment: 'prod'
    }
  }
  post {                              # No direct equivalent
    failure {                         # Use conditions:
      mail to: 'team@...'            # condition: failed()
    }
  }
}
```

Key Concepts Mapping

Jenkins	Azure Pipelines	Notes
Jenkinsfile	azure-pipelines.yml	Both are pipeline-as-code
Agent/Node	Pool/Agent	Azure has Microsoft-hosted + self-hosted
Stage	Stage	Same concept
Step	Step/Task	Azure has pre-built Tasks from marketplace
Shared Library	Templates	Azure uses `extends` and `template` keywords
Credentials	Service Connections	Azure integrates with Key Vault
Post actions	Conditions	`condition: always()`, `condition: failed()`
Multibranch	Trigger with branch filters	Similar behavior

Top Interview Questions

Q: "What is the difference between a Microsoft-hosted and self-hosted agent?"

Feature	Microsoft-Hosted	Self-Hosted
Provisioning	Auto — fresh VM each run	You manage the VM/container
Cost	Included (limited free mins)	Your infra cost
Customization	Limited to pre-installed tools	Full control
Network	Public internet	Can access private networks
Security	Managed by Microsoft	Your responsibility
When to use	Simple builds, public repos	Private networks, custom tools, compliance

> ***Your answer:*** *"This is similar to Jenkins. Microsoft-hosted agents are like using a cloud-based Jenkins agent — fresh environment each time, no maintenance. Self-hosted agents are like our on-premise Jenkins agents — we control the tooling, they can access internal networks, and we handle patching. I'd use self-hosted when the pipeline needs to access private resources like internal registries or databases."*

Q: "Explain stages, jobs, steps, and tasks"

```
Pipeline
├── Stage 1 (Build)           ← Logical boundary, can have approvals
│   ├── Job 1 (Compile)       ← Runs on ONE agent, parallel with other jobs
│   │   ├── Step 1 (Checkout) ← Individual action
│   │   ├── Step 2 (Build)
│   │   └── Step 3 (Test)
│   └── Job 2 (Lint)          ← Runs in parallel with Job 1
│       └── Step 1 (Run ESLint)
├── Stage 2 (Deploy Staging)  ← Depends on Stage 1
│   └── Deployment Job        ← Special job type with environment + strategy
└── Stage 3 (Deploy Prod)     ← Manual approval before this stage
    └── Deployment Job
```

Q: "How do you implement approvals in Azure Pipelines?"

> *"Approvals are configured on Environments, not on the pipeline itself. You create an Environment (e.g., 'production') in Azure DevOps, then add Approval checks — requiring specific people or groups to approve before any deployment job targeting that environment can proceed. This is like Jenkins' `input` step but more structured and auditable. You can also add business hours checks, exclusive locks, and branch control."*

Chapter 10: Artifacts, Service Connections, Variable Groups

Service Connections (Probability: 🟠 70%)

> ***Jenkins equivalent:*** *Credentials (stored in Jenkins Credential Store)*

Q: "What is a Service Connection and how do you secure it?"

> *"A Service Connection is how Azure Pipelines authenticates to external services — Azure subscriptions, Docker registries, Kubernetes clusters, AWS, etc. It's like a Jenkins credential but with RBAC at the project level. Best practices: 1) Use Workload Identity Federation (OIDC) instead of client secrets — no secrets to rotate. 2) Scope to specific resource groups using least-privilege. 3) Restrict which pipelines can use it. 4) Audit usage regularly."*

Variable Groups (Probability: 🟡 60%)

> ***Jenkins equivalent:*** *Environment variables + Credentials combined*

```
# Referencing a variable group in YAML
variables:
- group: 'production-secrets'     # Linked from Azure DevOps Library
- name: buildConfiguration
  value: 'Release'

# Variable group can be linked to Azure Key Vault
# Secrets are fetched at runtime — never stored in pipeline
```

Q: "How do you manage environment-specific configuration?"

> *"I use Variable Groups — one per environment (dev, staging, prod). Sensitive values are linked to Azure Key Vault so secrets are fetched at runtime and never stored in the pipeline definition. Non-sensitive config (like API URLs, feature flags) goes directly in the variable group. This is cleaner than Jenkins where you'd typically manage this through a combination of credential stores, environment variables, and parameterized builds."*

Azure Artifacts (Probability: 🟡 40%)

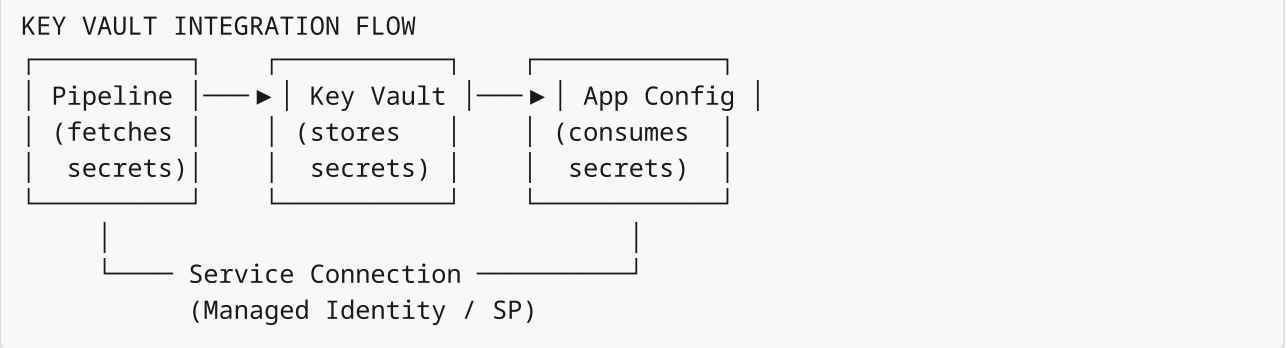
> ***Jenkins equivalent:*** *Nexus/Artifactory*

Q: "What is Azure Artifacts?"

> *"It's a package management service — hosts npm, Maven, NuGet, pip, and universal packages. Think of it as a private Nexus/Artifactory built into Azure DevOps. Key use: host internal packages that your pipelines consume. Supports upstream sources (proxy to public registries). Useful for caching public packages to avoid rate limiting and for hosting internal shared libraries."*

Chapter 11: Azure Key Vault, ACR, AKS

Azure Key Vault (Probability: 🟡 55%)



Q: "How does Azure Key Vault compare to what you use now?"

> *"Currently we use Kubernetes Secrets (encrypted at rest) and Jenkins Credential Store. Key Vault is more enterprise-grade — it provides centralized secret management with audit logging, automatic rotation, soft-delete protection, RBAC, and integration with Azure services. The equivalent in AWS is AWS Secrets Manager or SSM Parameter Store. In our setup, I'd link Key Vault to Variable Groups so pipelines fetch secrets at runtime without them ever being stored in pipeline code."*

Azure Container Registry (ACR) (Probability: 🟡 50%)

> ***Your equivalent:*** *Self-hosted GitLab Container Registry or DockerHub*

Q: "What's ACR and how would you integrate it?"

> *"ACR is Azure's private Docker registry — equivalent to our self-hosted GitLab registry. Key advantages: geo-replication for multi-region deployments, integrated vulnerability scanning with Microsoft Defender, content trust (image signing), and native integration with AKS (no ImagePullSecrets needed if using managed identity). In a pipeline: build image → push to ACR → AKS pulls from ACR using managed identity."*

Azure Kubernetes Service (AKS) (Probability: 🟡 55%)

> ***Your equivalent:*** *Self-managed Kubernetes clusters*

Q: "AKS vs self-managed Kubernetes — what's different?"

Feature	Self-Managed K8s	AKS
Control plane	You manage (etcd, API server, scheduler)	Azure manages (free)
Node management	You provision, patch, scale	Node pools, auto-scale, auto-upgrade
Networking	You configure CNI	Azure CNI or Kubenet built-in
Monitoring	Prometheus + Grafana (you build)	Azure Monitor + Container Insights
Security	You manage RBAC, policies	AAD integration, Azure Policy
Cost	Infra only	Infra only (control plane free)

> ***Your answer:*** *"My Kubernetes fundamentals are identical — deployments, services, ingress, RBAC, HPA, PVCs — the K8s API is the same. The difference is operational: AKS abstracts the control plane. I don't manage etcd backups or API server certificates. AKS adds Azure-specific features like AAD-integrated*

> *RBAC, Azure CNI networking, and node pool auto-scaling. My Prometheus+Grafana stack maps to Azure Monitor + Container Insights."*

Chapter 12: Jenkins (Your Existing Tool)

Probability: 🟠 80%

Declarative Pipeline

```
pipeline {
    agent {
        docker {
            image 'node:18-alpine'    // Run in container
            args '-v /tmp:/tmp'
        }
    }

    environment {
        REGISTRY = 'registry.example.com'
        IMAGE = "${REGISTRY}/myapp:${BUILD_NUMBER}"
    }

    options {
        timeout(time: 30, unit: 'MINUTES')
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                sh 'npm ci'
                sh 'npm run build'
            }
        }

        stage('Test') {
            parallel {                  // Parallel execution
                stage('Unit Tests') {
                    steps { sh 'npm test' }
                }
                stage('Lint') {
                    steps { sh 'npm run lint' }
                }
            }
```

```
        }

        stage('Security Scan') {
            steps {
                sh "grype ${IMAGE} --fail-on critical"
            }
        }

        stage('Deploy') {
            when {
                branch 'main'           // Only deploy from main
            }
            input {                      // Manual approval
                message 'Deploy to production?'
                ok 'Yes, deploy'
            }
            steps {
                sh "kubectl apply -f k8s/"
            }
        }
    }

    post {
        failure {
            slackSend channel: '#alerts',
                message: "Build FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
        }
        always {
            cleanWs()                   // Clean workspace
        }
    }
}
```

Key Interview Questions

Q: "Declarative vs Scripted Pipeline?"

Feature	Declarative	Scripted
Syntax	`pipeline { }` block	`node { }` block
Structure	Predefined structure	Groovy flexibility
Error handling	`post { }` block	`try/catch/finally`
Validation	Pre-validated before run	Runtime errors possible
Recommendation	**Use for most cases**	Complex logic only

Q: "How do you handle Jenkins high availability?"

> *"Jenkins itself doesn't support active-active HA natively. Our approach: 1) Regular automated backups of JENKINS_HOME to S3/Object Storage, 2) Configuration-as-code plugin to rebuild from code, 3)*

> *Separate build agents from the master — agents are ephemeral Docker containers, 4) For critical pipelines, we have GitLab CI as a failover option."*

Q: "How do you manage Jenkins credentials securely?"

> *"We use the Jenkins Credential Store with folder-level scoping. Secrets are never printed in logs (masked automatically). For cloud access, we use IAM roles attached to the Jenkins agent instances rather than storing access keys. We rotate credentials quarterly and audit credential usage through the audit trail plugin."*

Jenkins Troubleshooting Scenarios

Scenario	Diagnosis	Fix
Build stuck, no logs	Agent disconnected	Check agent connectivity, restart agent
"No space left on device"	Workspace not cleaned	Add `cleanWs()` in post, increase disk, prune old builds
Pipeline hangs at `input`	Waiting for approval	Approve or add timeout: `timeout(time: 1, unit: 'HOURS')`
Plugin compatibility issue	After Jenkins/plugin update	Pin plugin versions, test upgrades in staging first
Concurrent build conflicts	Shared resources	Use `disableConcurrentBuilds()` or resource locks
Slow builds	Large workspace, no caching	Use pipeline caching, docker layer caching, parallel stages